

ICT 1306

OBJECT ORIENTED PROGRAMMING

4.OBJECTS & CLASSES

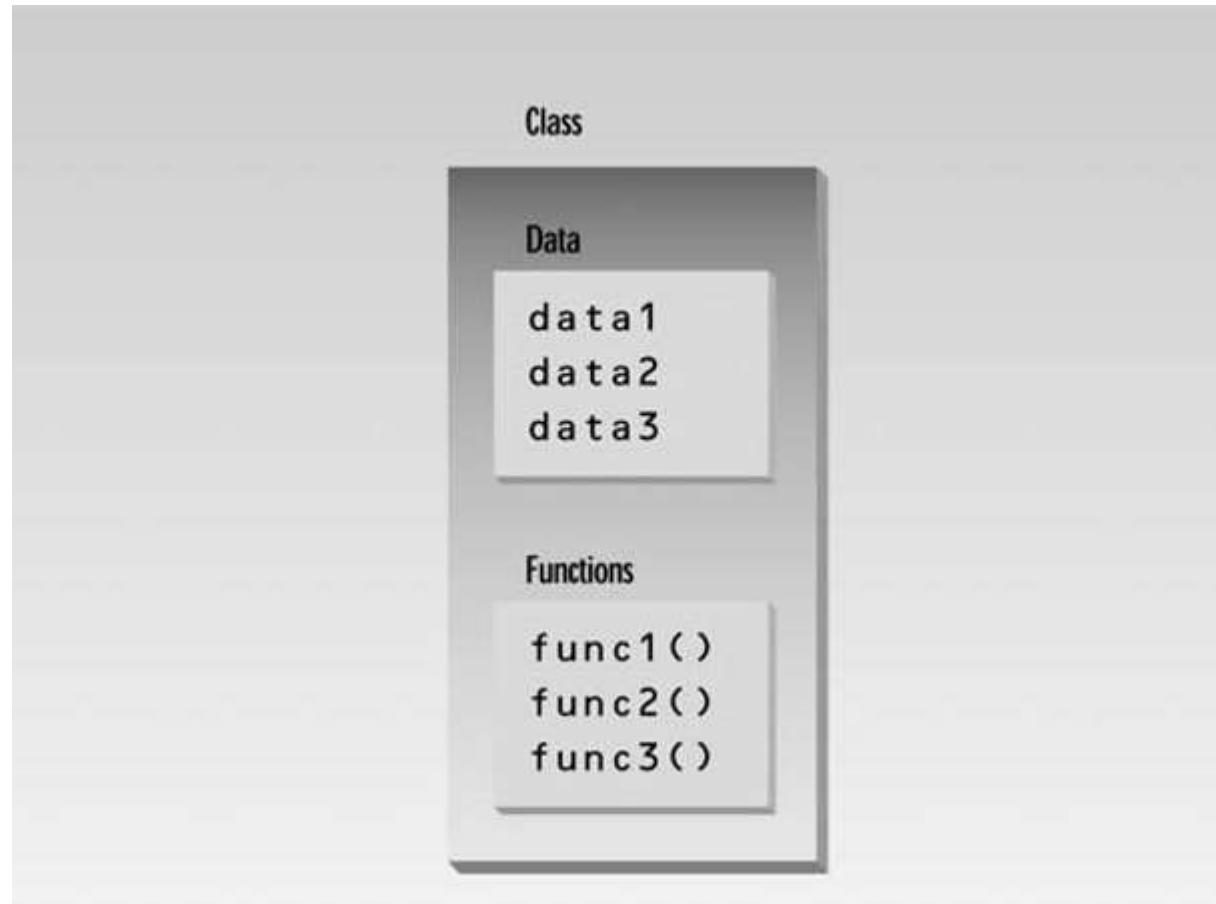
TC Irugalbandara
KHA Hettige

Syntax of a Class in C++

```
// smallobj.cpp
// demonstrates a small, simple object
#include <iostream>
using namespace std;
class smallobj      //define a class
{
    private:
        int somedata;    //class data
    public:
        void setdata(int d) //member function to set data
        { somedata = d; }
        void showdata()    //member function to display data
        { cout << "Data is " << somedata << endl; }
};
```

```
int main() {
    smallobj s1, s2;
    //define two objects of class smallobj
    s1.setdata(1066);
    //call member function to set
    data
    s2.setdata(1776);
    s1.showdata();
    //call member function to display data
    s2.showdata();
    return 0;
}
```

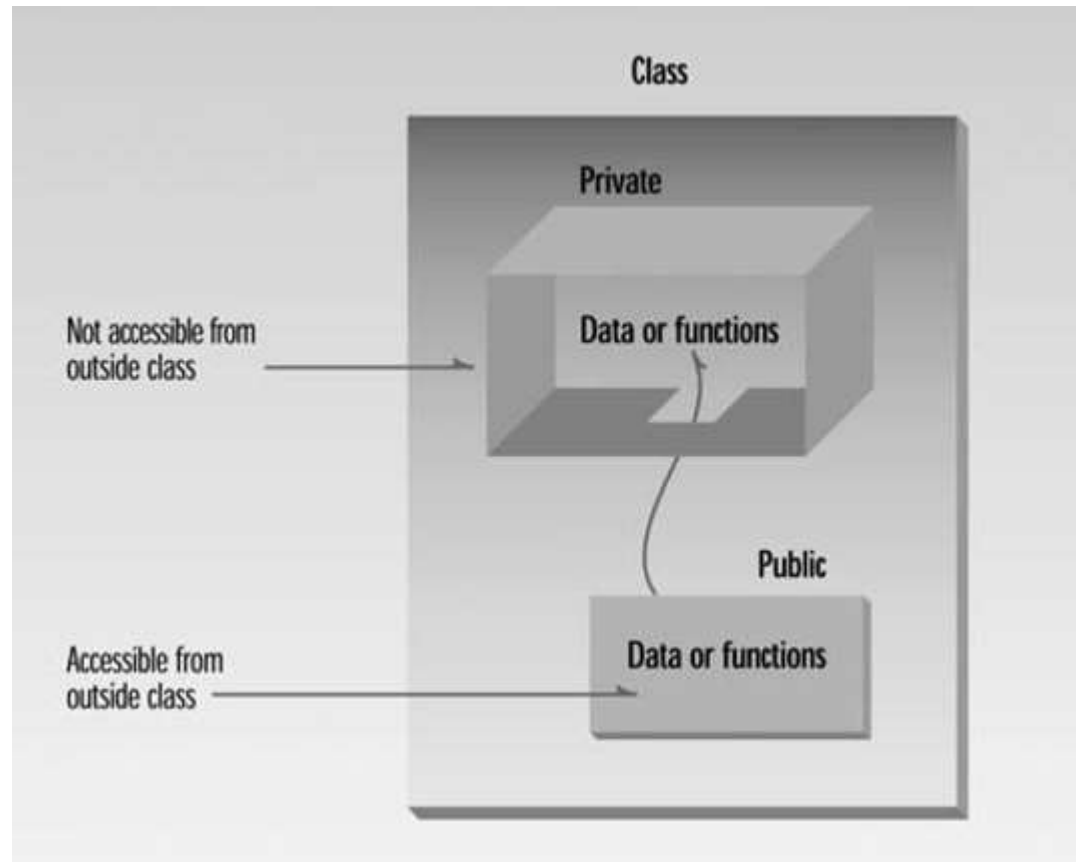
- Placing data and functions together into a single entity is a central idea in object-oriented programming



Classes and Objects

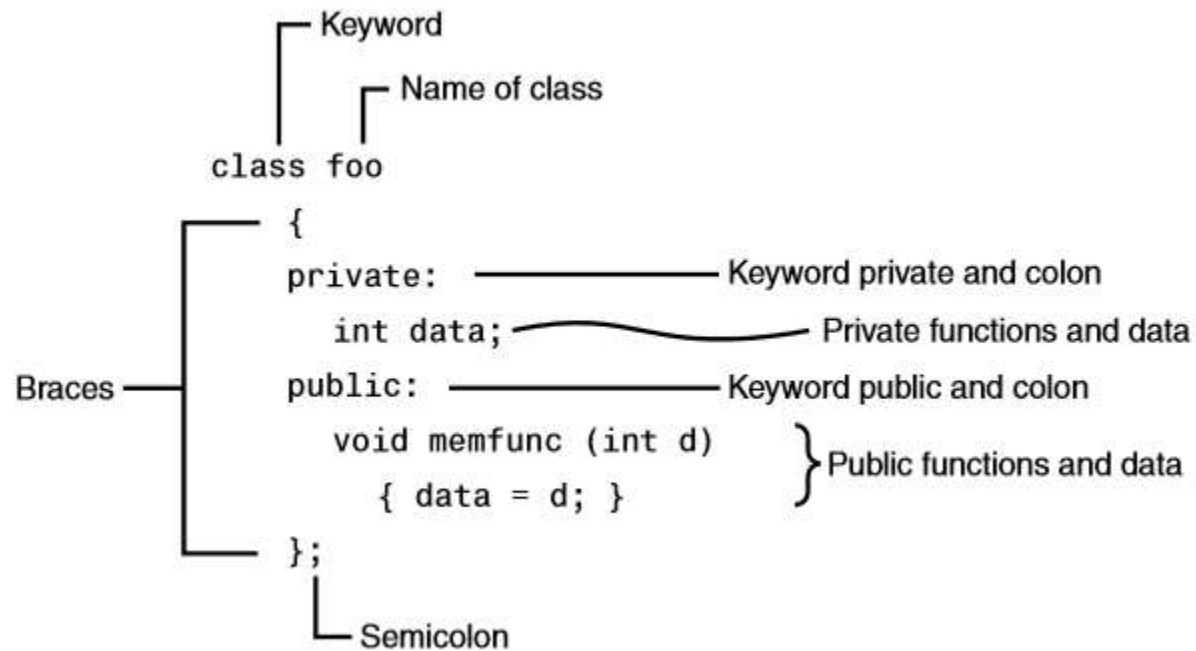
Class keyword Class name

```
class smallobj      //define a class
{
    private:         ← Data Hiding
    int somedata;    //class data
    public:          ← Can access from outside the class
    void setdata(int d) //member function to set data
    { somedata = d; }
    void showdata()   //member function to display data
    { cout << "Data is " << somedata << endl; }
};                  ← Semicolon mark the end of the construct
```



- The data items within a class are called data members (or sometimes member data).
- Member functions are functions that are included within a class.
- Usually the **data within a class is private** and the **functions are public** .

Syntax of a Class Definition



Using the Class

- Defining the Objects
 - `smallobj s1, s2;`
 - Space is set aside for it in memory.
 - **Instantiating**
- Calling the Member Functions
 - `s1.setdata(189);`
 - `s2.setdata(67);`

Example

```
#include <iostream>

using namespace std;

class book      //define class
{
    private:
        int bookID;
        //ID number of widget
        int num_of_copies;
        //ID number of widget part
        float price;    //cost of part
```

```
};
```

```
public:
    void setbook(int mn, int pn, float c)
    //set data
    {
        bookID = mn;
        num_of_copies = pn;
        price = c;
    }
    void showbook()
    //display data
    {
        cout << "BookID" << bookID;
        cout << ", Number of Copies " << num_of_copies;
        cout << ", Unit Price $" << Price << endl;
    }
}
```

```
int main() {  
    book book1;           //define object of class part  
    book1.setbook(1234, 50, 350.50F); //call member function  
    book1.showbook();      //call member function  
    return 0;  
}
```

Q1.

- Create a class call "student"
- Data members : Name(String);Age(Int);Registration Number(Int),Gender (Char)
- Member Functions : add student; show student
- Create 3 objects of student type.
- Initialize 2 student objects with constant values, take data from the user for 1 object.
- Display all the details.

Q2. Modify the above programme to have 3 member functions

- setstudent→ which uses arguments to set student data;
- getstudent()→which gets values from the user at the keyboard; and
- showstudent()→ which displays the details.

- Write a programme to create Distance objects. Write methods to get distance, set distance and show distance.

```
#include <iostream>
using namespace std;
class Distance          // Distance class
{
private:
    int feet;
    float inches;
public:
    void setdist(int ft, float in) //set Distance to args
    {
        feet = ft;
        inches = in;
    }
}
```

```
void getdist()      //get length from user
```

```
{  
    cout << "\nEnter feet: ";  
    cin >> feet;  
    cout << "Enter inches: ";  
    cin >> inches;
```

```
}
```

```
void showdist()     //display distance
```

```
{  
    cout << feet << "'-" << inches << "'"; }  
};
```

```
int main() {  
    Distance dist1, dist2;    //define two lengths  
    dist1.setdist(11, 6.25);  //set dist1  
    dist2.getdist();          //get dist2 from user  
    //display lengths  
    cout << "\ndist1 = "; dist1.showdist();  
    cout << "\ndist2 = "; dist2.showdist();  
    cout << endl;  
    return 0;  
}
```

Constructors

- It's convenient if an object can initialize itself when it's first created, without requiring a separate call to a member function.
- Automatic initialization is carried out using a special member function called a **constructor**.
- A constructor is a member function that is executed automatically whenever an object is created.
- Constructors can be very useful for setting initial values for certain member variables.

```
// counter.cpp
// object represents a counter variable
#include <iostream>
using namespace std;
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
class Counter
{
private:
    unsigned int count;                //count
public:
    Counter() : count(0)                //constructor
    { /*empty body*/ }
    void inc_count()                    //increment count
    { count++; }
    int get_count()                    //return count
    { return count; }
};
```

```
int main()
{
    Counter c1, c2;                //define and initialize

    cout << "\nc1=" << c1.get_count(); //display
    cout << "\nc2=" << c2.get_count();

    c1.inc_count();                //increment c1
    c2.inc_count();                //increment c2
    c2.inc_count();                //increment c2

    cout << "\nc1=" << c1.get_count(); //display again
    cout << "\nc2=" << c2.get_count();

    cout << endl;
    return 0;
}
```

- Each object initialize itself when it is created.
- Same name as class. No return type (compiler identify as a constructor).
- The initialization takes place following the member function declarator but before the function body. It's preceded by a colon. The value is placed in parentheses following the member data.

```
count() : count(0)
{ }
```

- If multiple members must be initialized, they're separated by commas. The result is the initializer list.

```
someClass() : m1(7), m2(33), m2(4) ← initializer list
{ }
```

- If we modify the above constructor as follows . What will be the output???

```
Counter() : count(0)
```

```
{ cout << "I'm the constructor\n"; }
```

Destructors

- Another function is called automatically when an object is destroyed. That is called a destructor. To deallocate the memory.
- A destructor has the same name as the constructor (which is the same as the class name) but is preceded by a tilde.
- They also take no arguments .No return type.

```
class Student {  
    private:  
    int data;  
    public:  
    Student() : data(0)    //constructor (same name as class)  
    { }  
    ~Student()            //destructor (same name with tilde)  
    { }  
};
```

Overloaded Constructors

```
#include <iostream>
using namespace std;
class Distance
{
    private:
        int feet;
        float inches;
    public:
        Distance() : feet(0), inches(0.0) { }
        Distance(int ft, float in) : feet(ft), inches(in) { }
        void getdist() {
            cout << "\nEnter feet: "; cin >> feet; cout << "Enter inches: "; cin >> inches;
        }
        void showdist() {
            cout << feet << "'-" << inches << "'";
        }
        void add_dist( Distance, Distance ); //declaration
};
```

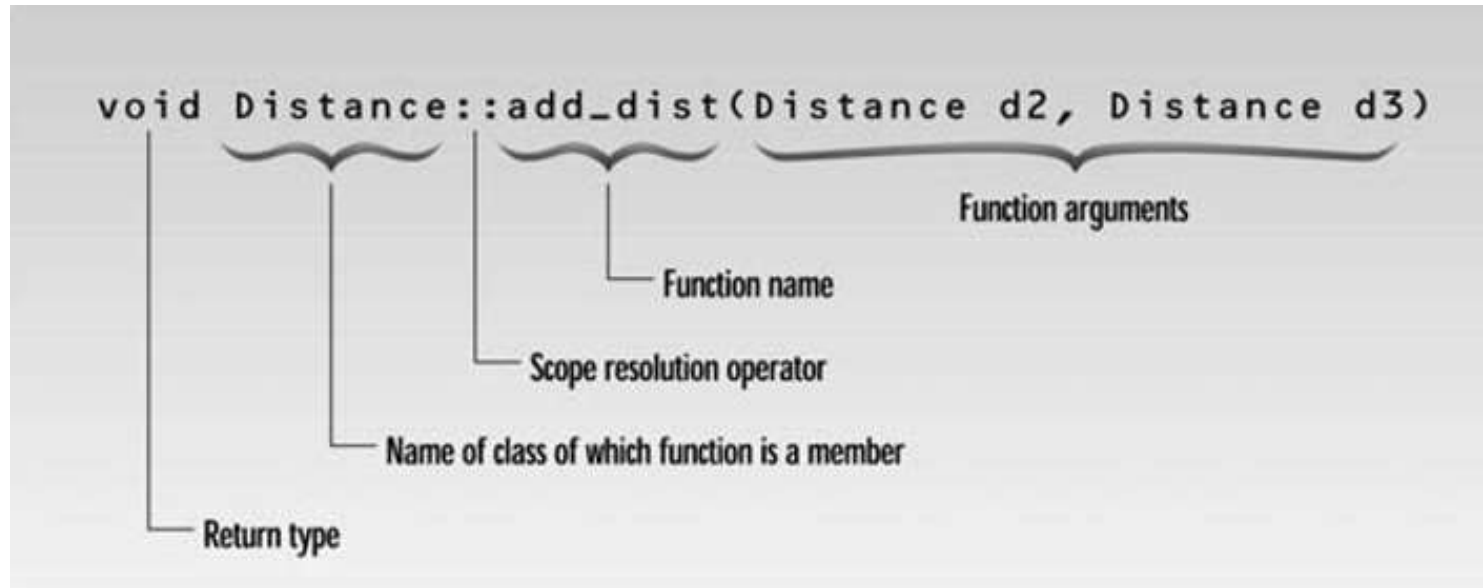
```
void Distance::add_dist(Distance d2, Distance d3)
{
    inches = d2.inches + d3.inches;
    feet = 0;
    if(inches >= 12.0) {
        inches -= 12.0;
        feet++;
    }
    feet += d2.feet + d3.feet;
}
```

```
int main() {  
    Distance dist1, dist3;    //define two lengths  
    Distance dist2(11, 6.25); //define and initialize dist2  
    dist1.getdist();          //get dist1 from user  
    dist3.add_dist(dist1, dist2); //dist3 = dist1 + dist2  
    cout << "\ndist1 = "; dist1.showdist();  
    cout << "\ndist2 = "; dist2.showdist();  
    cout << "\ndist3 = "; dist3.showdist();  
    cout << endl;  
    return 0;  
}
```

Default Constructor

- Implicit no-argument constructor is built into the program automatically by the compiler.
- This constructor creates the objects, even though we didn't define it in the class.
- This no-argument constructor is called the default constructor.
- Which of the two constructors is executed when an object is created in the above example, depends on how many arguments are used in the definition.

Scope Resolution Operator



- Member functions need not to be defined inside the class definition.
- Can declare inside the class and define outside.
- The double colon (`::`) is a symbol called the scope resolution operator. It is a way of specifying what class something is associated with.

Default Copy Constructor

- Object can be initialized with another object of the same type.
- No need to create a special constructor for this; one is already built into all classes. It's called the default **copy constructor**.
- It's a one argument constructor whose argument is an object of the same class as the constructor.

Ex:

```
class Distance                                //English Distance class
{
private:
    int feet;
    float inches;
public:
    //constructor (no args)
    Distance() : feet(0), inches(0.0)
    { }
    //Note: no one-arg constructor
    //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }

    void getdist()                            //get length from user
    {
        cout << "\nEnter feet: ";  cin >> feet;
        cout << "Enter inches: ";  cin >> inches;
    }
    void showdist()                          //display distance
    { cout << feet << "'-" << inches << "'"; }
};
```

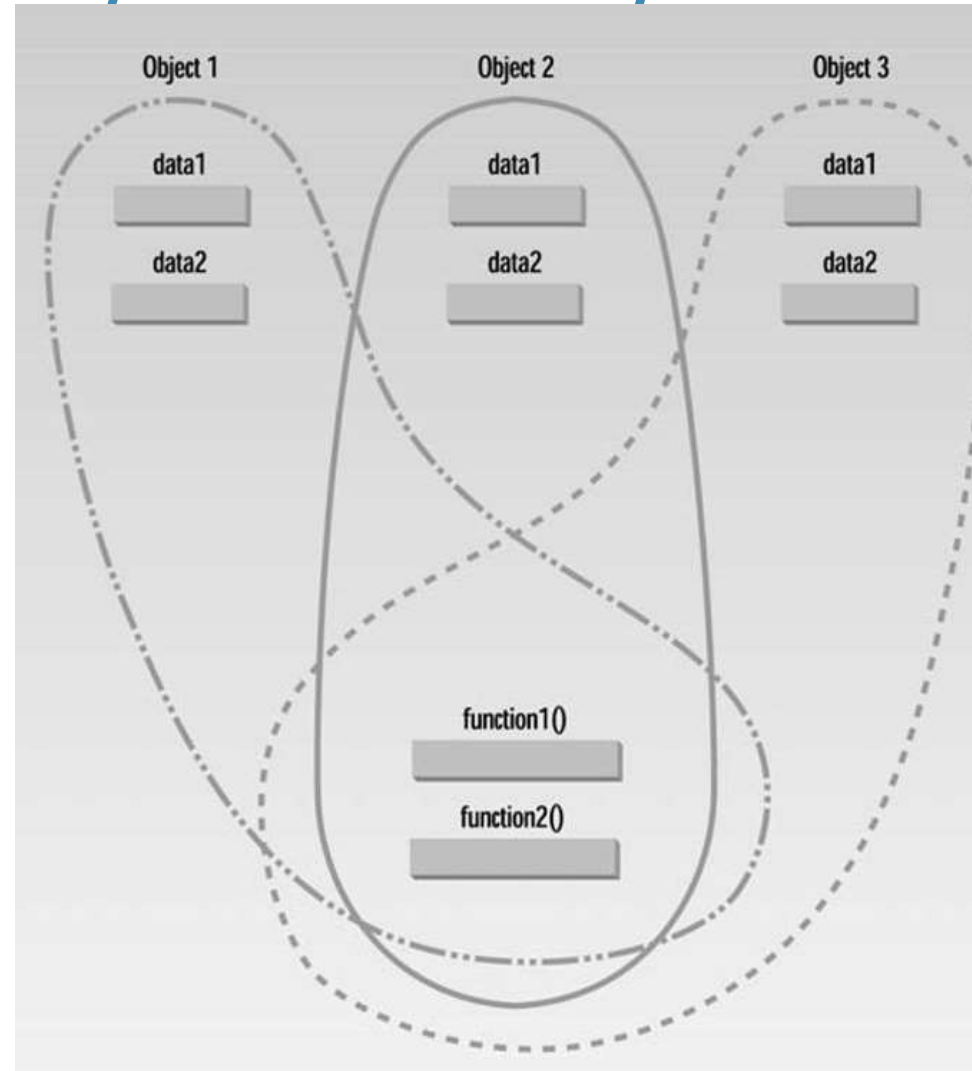
```
int main()
{
    Distance dist1(11, 6.25);    //two-arg constructor
    Distance dist2(dist1);       //one-arg constructor
    Distance dist3 = dist1;      //also one-arg constructor

                                //display all lengths
    cout << "\ndist1 = "; dist1.showdist();
    cout << "\ndist2 = "; dist2.showdist();
    cout << "\ndist3 = "; dist3.showdist();
    cout << endl;
    return 0;
}
```

Returning Objects from Functions

- Objects can be passed as arguments to functions. Same as we can create functions that return Objects.
- Q1. Modify the `add_dist()` function in Distance class to return a Distance Object.

Objects, Data, Functions, and Memory



Static Data

- If a data item in a class is declared as **static**, only one such item is created for the entire class, no matter how many objects there are.
- A static data item is useful when all objects of the same class must share a common item of information.
- A member variable defined as static has characteristics similar to a normal static variable: It is visible only within the class, but its lifetime is the entire program.

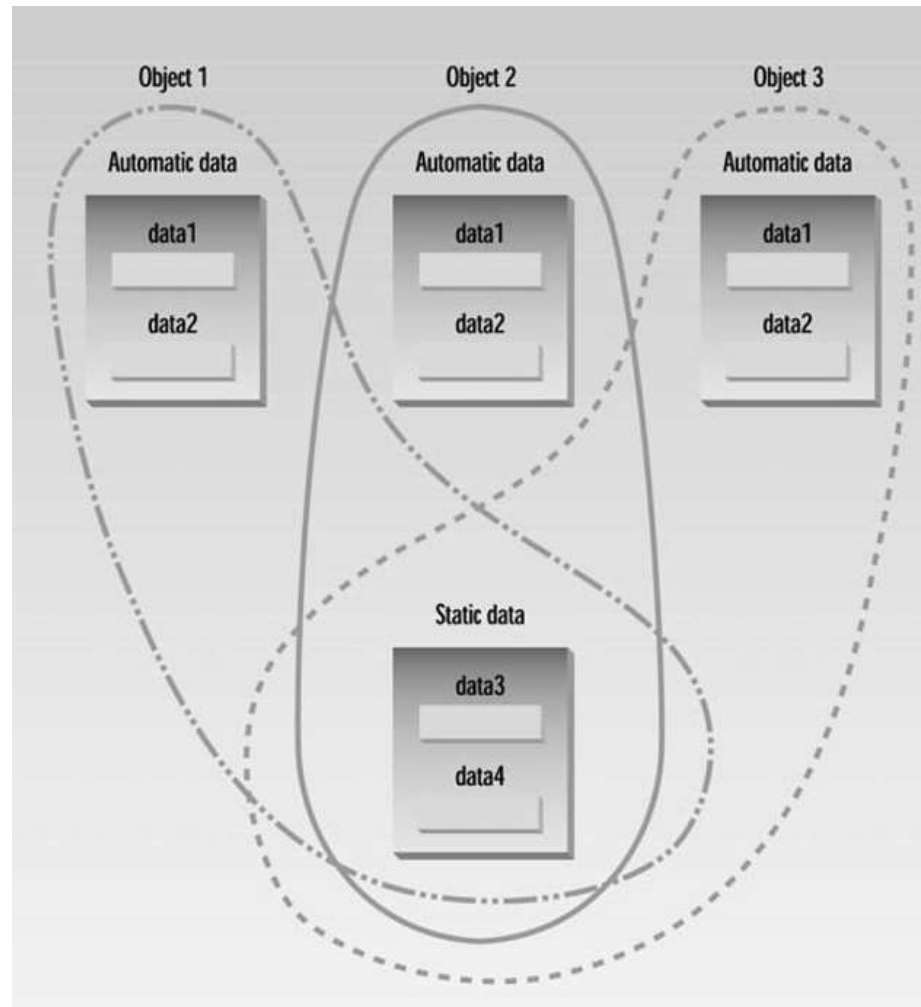
Example

```
// objpart.cpp
// widget part as an object
#include <iostream>
using namespace std;
class book      //define class
{
    private:
        static int count; // declaration
        int bookID;
        int num_of_copies;
        float price;
```

```
public:
    book() { count++; }
    void setbook(int mn, int pn, float c)
    {
        bookID = mn;
        num_of_copies = pn;
        price = c;
    }
    void showbook()
    {
        cout << "BookID" << bookID;
        cout << ", Number of Copies " << num_of_copies;
        cout << ", Unit Price $" << Price << endl;
        cout<<"Number of Books:"<<count;
    }
};

int book::count = 0; // Definition of count
```

Static Vs. Automatic Data



const Member Functions

- A **const** member function guarantees that it will never modify any of its class's member data.

```
class Book{  
    private :  
        Int price;  
    public:  
    void fun1()  
        { price=100;} // Correct  
    void fun2() const  
        { price=100;} // Error because const can't modify a member (Read Only Structures)  
  
};
```

const Member Function Arguments

- If you pass an argument to a member function and if you don't want the function to modify it, the argument should be made const in the function declaration and definition.

```
class Book{
    private :
        Int price;
    public:
        void fun1()
            { price=100;} // Correct
        void fun2() const
            { price=100;} // Error because const can't modify a member (Read Only Structures)
        void increase_price(const Book&) const;
};
```

```
void Book::increase_price(const Book& b) const
{
    Book b1;
    price=120; //ERROR
    b.price=120; //ERROR
    b1.price=120;
}

int main(){
    /* Create objects of type Book and Check above */

}
```

const Objects

- We can apply const to objects of classes.
- When an object is declared as const, you can't modify it.
- And also you can use only const member functions with it, because they're the only ones that guarantee not to modify it.

```
class Book{
    private :
        Int price;
    public:
        void fun1()
            { price=100;} // Correct
        void fun2() const
            { price=150;} // Error because const can't modify a member (Read Only Structures)

        void increase_price(const Book&) const;
};
```

```
void Book::increase_price(const Book& b) const
{
    Book bb;
    bb.price=120;
    cout<<b.price;

}

int main(){
    const Book b1
    b1.fun1(); // 1
    b1.fun2(); // 2
    b1.increase_price(b1); // 3
}
```

What should be 1,2,3.

Try it!

Develop a system to automate the functions at a tollbooth.

- Each vehicle passing by the booth are expected to pay a Rs.50(amount per 1km) per 1km to toll. But sometimes they go without paying. The tollbooth keeps track of the number of vehicles that have gone by, and of the total amount of money collected. Model this tollbooth with a class called tollBooth.
- The two data items are a type unsigned int to hold the total number of vehicles, and a type double to hold the total amount of money collected and no of kilometers they have driven, type int to hold the amount per 1 km which should not be change by any method.
- Initializes amount per km to Rs.50 and all others to 0.
- A member function called payingVehicle() increments the car total and adds payment to the cash total. Another function, called nopayVehicle(), increments the car total but adds nothing to the cash total. No of kilometers should be given by user. Finally, a member function called display() displays the two totals. Make appropriate member functions const.
- Include a program to test this class. Implement it to get user inputs until user give a command to stop. If need to stop then display the total.

Thank You !